

Alert Dialog Box

⇒ They are used to help users answer questions, make selections, confirm actions, and read warning or error messages. Dialog displays small windows.

⇒ it is a window that partially obscures the activity that launched it.

⇒ it can be used to display the dialog message with OK and Cancel buttons.

⇒ AlertDialog is composed of three regions:

- Title
- Content area
- action buttons

⇒ AlertDialog is the subclass of Dialog class.

⇒ Only three buttons are allowed in dialog box. if want to more then go to customize dialog box.

A popup which draws user attention over the activity before they continue moving forward.

3 Types of Alert Dialog Box

(1) Confirmation

- Terms and Conditions
- One button

(2) Delete:

- Are you sure you want to delete?
- Yes No
- Two buttons

(3) Exit:

- Exit
- Cancel No Yes
- Three buttons.

AlertDialog vs AlertDialog.Builder:

Object	Return type
◦ new AlertDialog.Builder (MainActivity.this).create()	for single Button
◦ new AlertDialog.Builder (MainActivity.this)	for 2 or 3 buttons.

Methods of AlertDialog class:

(1) Public AlertDialog.Builder setTitle (charSequence):

o This method is used to set the title of AlertDialog.

(2) Public AlertDialog.Builder setMessage (charSequence):

o This method is used to set the message for AlertDialog.

(3) public AlertDialog.Builder setIcon (int):

o This method is used to set the icon over AlertDialog.

Example:

For Single Button:

Write code in onCreate method.

```
AlertDialog alertDialog = new AlertDialog.  
Builder (context: this).create();
```

```
alertDialog.setTitle ("Terms & Conditions");  
alertDialog.setIcon ("R.drawable.ic_baseline  
-info-24");
```

```
alertDialog.setMessage ("Have you all  
the T and C");
```

```
alertDialog.setPositiveButton ("Yes, I've Read", new  
DialogInterface.OnClickListener() {  
    @Override
```

```
        public void onClick(DialogInterface  
dialogInterface, int i)  
        {
```

```
            Toast.makeText (MainActivity.this,  
"Yes, you can proceed now...", Toast.LENGTH  
SHORT).show();  
        }  
    });
```

```
alertDialog.show();
```

For Two buttons:

```
AlertDialog.Builder delDialog = new  
AlertDialog.Builder (MainActivity.this);
```

```
delDialog.setTitle ("Delete?");
```

```
delDialog.setIcon ("R.drawable.ic_baseline  
delete-24");
```



```
delDialog.setMessage("Are you want  
to delete?");
```

```
delDialog.setPositiveButton("Yes", new DialogInterface.  
onClickListener() {
```

```
@Override
```

```
public void onClick(DialogInterface  
dialogInterface, int i)  
{
```

```
Toast.makeText(MainActivity.this, "Item Delete",  
Toast.LENGTH_SHORT).show();
```

```
}
```

```
});
```

```
delDialog.setNegativeButton("No", new DialogInterface.  
onClickListener() {
```

```
@Override
```

```
public void onClick(DialogInterface  
dialogInterface, int i)  
{
```

```
Toast.makeText(MainActivity.this,  
"Item Not Deleted", Toast.LENGTH_SHORT).show();  
}
```

```
}); delDialog.show(); }
```

press three Button:

```
@Override
```

```
public void onBackPressed()  
{
```

```
AlertDialog.Builder exitDialog = new  
AlertDialog.Builder(this);
```

```
exitDialog.setTitle("Exit?");
```

```
exitDialog.setMessage("Are you want  
to exit?");
```

```
exitDialog.setIcon("R.drawable/ic_baseline  
exit_to_app_24");
```

```
exitDialog.setPositiveButton("No", new  
DialogInterface.OnClickListener() {
```

```
@Override
```

```
public void onClick(DialogInterface  
dialogInterface, int i) {
```

```
Toast.makeText(MainActivity.this, "Welcome  
back!", Toast.LENGTH_SHORT).show();  
}
```

```
});
```



```

exitDialog.setNegativeButton("Yes", new
DialogInterface.OnClickListener() {
    public void onClick(DialogInterface
dialogInterface, int i)
    {

```

```

        MainActivity.super.onBackPressed();
    }
});

```

```

exitDialog.setNeutralButton("Cancel"
"Operation Cancelled", Toast.LENGTH_SHORT).
show();
}

```

```

}); exitDialog.show();
}

```

Customizing Dialog Box

- In customizing dialog box - We use design a custom layout, inflating it in code, customizing its behavior, and displaying it to users.
- A customized layout is created for toast notification.

Custom Dialog VS Alert Dialog

Alert Dialog	Custom Dialog
<u>Predefined Layout:</u> ◦ Yes, predefined layout and style.	◦ No, developers define the layout themselves using XML files or programmatically.
<u>Customization Options:</u> ◦ Limited (title, message, buttons).	◦ Extensive (full control over layout and behavior).

Use Cases:

- Simple alerts
- Confirmations
- basic input

- Completeness of UI
- input forms
- advanced UX

Layout Design:

- Standard layout

- Custom layout
(xml or programming)

Styling

- Limited styling options provided by system.

- Full styling controls based on developer design.

Low, quick to implement.

Higher, requires more design and coding effort.

Less flexible due to predefined structure.

Highly flexible

Code: res > layout

<LinearLayout

android:background="#f0f0f0"

android:width="match-parent"

android:height="wrap-content"

android:orientation="vertical">

<ImageView

android:layout_width="match-parent"

android:layout_height="70dp"

android:src="@drawable/ic_baseline_done_24">

<LinearLayout

android:layout_width="match-parent"

android:layout_height="wrap-content"

android:background="#fff"

android:orientation="vertical">

<TextView

android:layout_width="match-parent"

android:layout_height="wrap-content"

android:text="Success"

android:textSize="34sp"

android:gravity="center"

android:textColor="#219642">

>

<TextView

android:layout_width="match-parent"

android:layout_height="wrap_content"

android:text="Your code has
been successfully placed!"

android:textsize="16sp".

textColor="#000" />

<LinearLayout>

<LinearLayout>

// Here you can define button.

Dialog dialog = new Dialog(this);

dialog setContentView(R.layout.custom_layout);

dialog.show();

// for Button

Button btnOkay = dialog.findViewById
(R.id.btnOkay);

btnOkay.setOnClickListener(new View.

OnClickListener() {

@Override

public void onClick(View view)

{

Toast.makeText(MainActivity.this, "Closed",

Toast.LENGTH_SHORT).show();

dialog.dismiss();

}

});

dialog.show();

}

if used prompt must need then set:

dialog.setCancelable(false);